VULNERABILITY ASSESSMENT REPORT

Target: Damn Vulnerable Web Application (DVWA)
Testing Tool: OWASP ZAP
Environment: Kali Linux
Author: Varshith S Patil
Date: March 2026
Project: 001

Table of Contents

1. Confidentiality Statement
2. Disclaimer
3. Contact Information
4. Assessment Overview
5. Methodology
6. Scope of Testing
7. Risk Severity Rating
8. Executive Summary
9. Attack Summary
10. Environment Setup
11. Vulnerability Findings
12. Remediation Recommendations
13.Conclusion

Confidentiality Statement

This document contains confidential information related to the security assessment conducted on the Damn Vulnerable Web Application (DVWA). The report is intended only for educational and research purposes. Unauthorized copying, distribution, or disclosure of this report is prohibited without permission from the author.

Disclaimer

A vulnerability assessment represents the security posture of the tested system at the time of testing. The findings and recommendations provided in this report are based on the tools and techniques used during the assessment period.

Assessment Overview

The vulnerability assessment was conducted from **February 23rd, 2026 to February 28th, 2026** to evaluate the security posture of the **Damn Vulnerable Web Application (DVWA)** hosted in a controlled testing environment.

The objective of this assessment was to identify potential security vulnerabilities, misconfigurations, and weaknesses in the web application using automated and manual security testing techniques. The testing was performed using **OWASP ZAP** on a **Kali Linux environment**, following industry best practices for web application security testing.

All testing activities were carried out based on established security standards and methodologies, including the **NIST SP 800-115 Technical Guide to Information Security Testing and Assessment** and the **OWASP Web Security Testing Guide (v4)**.

Phases of penetration testing activities include the following:

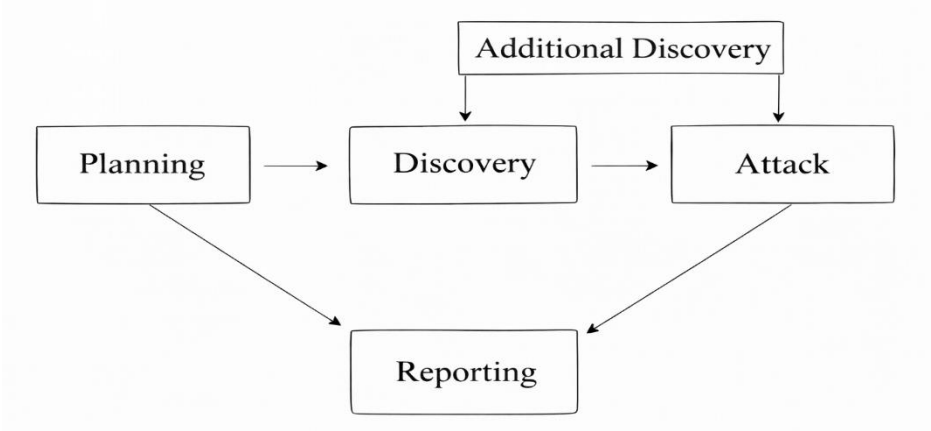
1. Planning: In this phase, the objectives of the assessment were defined and the scope of testing was established. The target application (**DVWA**) and testing environment were configured to ensure a controlled and safe testing process.

2. Discovery : During this phase, reconnaissance and vulnerability scanning were performed using **OWASP ZAP** to identify potential weaknesses, exposed directories, and missing security configurations within the web application.

3. Attack : In the attack phase, the discovered vulnerabilities were analyzed and validated to confirm their existence and potential impact on the application security.

4. Reporting: All identified vulnerabilities, observations, and security weaknesses were documented in this report. Recommendations were also provided to mitigate the identified security risks and improve the overall security posture of the application.

METHODOLOGY



Assessment Components

Vulnerability Assessment

The vulnerability assessment was conducted on the **Damn Vulnerable Web Application (DVWA)** to identify security weaknesses and misconfigurations in the web application environment. The testing was performed using **OWASP ZAP** in a controlled **Kali Linux environment**.

The purpose of this assessment was to simulate the actions of an attacker attempting to discover vulnerabilities in the application. The testing involved automated scanning and manual verification of discovered issues.

The following activities were performed during the assessment:

- **Application Enumeration** – Identifying accessible pages, directories, and application components.
- **Automated Vulnerability Scanning** – Using OWASP ZAP to detect security vulnerabilities such as missing security headers, information disclosure, and configuration weaknesses.
- **Manual Validation** – Reviewing the identified vulnerabilities to confirm their existence and determine their potential impact.
- **Security Analysis** – Evaluating how the discovered vulnerabilities could affect the confidentiality, integrity, and availability of the application.

Finding Severity Ratings

The following table defines the severity levels used in this report to classify identified vulnerabilities.

Severity	Description
High	Vulnerabilities that can significantly impact the security of the application and may allow attackers to compromise sensitive information.
Medium	Vulnerabilities that may expose the application to potential attacks but require additional conditions for exploitation.
Low	Minor security weaknesses that do not directly impact the system but should be fixed to improve overall security.
Informational	Security observations that do not represent direct vulnerabilities but provide useful informati about the application configuration.

Scope

Assessment	Details
Target Application	Damn Vulnerable Web Application (DVWA)
Testing Environment	Kali Linux
Testing Tool	OWASP ZAP
Access Method	Localhost
Target URL	http://127.0.0.1/login.php
Assessment Period	February 23 – February 28, 2026

Executive Summary

A vulnerability assessment was conducted on the **Damn Vulnerable Web Application (DVWA)** between **February 23, 2026 and February 28, 2026** to evaluate the security posture of the web application and identify potential security weaknesses. The assessment was performed in a controlled testing environment using **OWASP ZAP** on **Kali Linux**. The testing methodology followed industry-recognized standards including the **OWASP Web Security Testing Guide** and the **NIST SP 800-115 Technical Guide to Information Security Testing and Assessment**.The primary objective of this assessment was to identify vulnerabilities, security misconfigurations, and missing security controls that could potentially be exploited by attackers. Automated scanning and manual analysis were performed to detect weaknesses in the web application.

During the assessment, several vulnerabilities were identified including **missing HTTP security headers, directory browsing exposure, information disclosure issues, and insecure configuration settings**. Although DVWA is intentionally designed to be vulnerable for educational purposes, the findings highlight common security weaknesses that are often observed in real-world web applications.

Total records generated	High	Medium Severity	Low Severity	Informational
12	0	0	2	0

Attack Summary

Sl. no	Action	Recommendation	Severity
1	Content Security Policy (CSP) Header Not Set	Configure a Content Security Policy header to restrict the sources from which content can be loaded.	Medium
2	Directory Browsing Enabled	Disable directory listing on the web server to prevent attackers from viewing sensitive files and directories.	Medium
3	Missing Anti-Clickjacking Header	Implement **X-Frame-Options** or **Content-Security-Policy frame-ancestors** to prevent clickjacking attacks.	Medium

4	HTTP Only Site (No HTTPS)	Implement HTTPS using SSL/TLS encryption to protect data in transit	Medium
5	Cookie Without HttpOnly Flag	ChaConfigure cookies with the **HttpOnly** attribute to prevent client-side script access.	Low
6	Cookie Without SameSite Attribute	Set the **SameSite** cookie attribute to protect against Cross-Site Request Forgery (CSRF) attacks.	Low
7	X-Content-Type-Options Header Missing	Configure the **X-Content-Type-Options: nosniff** header to prevent MIME-type sniffing attacks.	Low
8	Server Version Information Disclosure	Hide server version information in HTTP headers to reduce the risk of targeted attacks.	Low
9	Security Configuration Weakness Identified	Apply secure server configuration and regularly update the system to reduce vulnerabilities.	Information al

Penetration test findings

1) Content Security Policy (CSP) Header Not Set (Medium)

Description	The web application does not implement a **Content Security Policy (CSP)** header. CSP is a security feature that helps prevent attacks such as cross-site scripting (XSS) and data injection by restricting the sources from which content can be loaded. Without a properly configured CSP header, the application becomes more vulnerable to malicious script execution.
Impact	An attacker may inject malicious scripts into the web application, which could lead to data theft, session hijacking, or unauthorized actions performed on behalf of the user.
System	http://127.0.0.1/dvwa

Proof of concept:

The vulnerability was identified during the OWASP ZAP automated scan, which detected the absence of the CSP header in the HTTP response.

Recommendation:- Configure a **Content Security Policy header** on the web server to restrict unauthorized scripts and external resources.

2) Directory Browsing Enabled (Medium)

Description	Directory browsing is enabled on the web server. When directory listing is enabled, users can view the contents of directories if an index file is not present.
Impact	Attackers may gain access to sensitive files such as configuration files, backups, or logs. This information may assist attackers in further exploitation of the application.
System	http://127.0.0.1/

Proof of concept:

The vulnerability was detected by OWASP ZAP during the crawling process, which identified accessible directory listings.

Recommendation:-

Disable directory listing in the web server configuration to prevent unauthorized viewing of files and directories.

3) Missing Anti-Clickjacking Header (Medium)

Description	The application does not implement an **X-Frame-Options** or **Content-Security-Policy frame-ancestors** header. This makes the application vulnerable to **clickjacking attacks**, where attackers embed the webpage within an invisible iframe on a malicious website.
Impact	Attackers can trick users into performing unintended actions such as clicking hidden buttons or submitting sensitive information.
System	http://127.0.0.1

Proof of concept:

OWASP ZAP detected that the anti-clickjacking header was missing from the HTTP response.

Recommendation:-

Configure the following header on the web server:

X-Frame-Options: DENY

or implement **Content-Security-Policy frame-ancestors**.

4) HTTP Only Site (No HTTPS) (Medium)

Description	The application is accessible over **HTTP instead of HTTPS**, meaning that communication between the client and server is not encrypted.
Impact	Sensitive data such as login credentials or session cookies may be intercepted by attackers through **Man-in-the-Middle (MITM) attacks**.
System	http://127.0.0.1

Proof of concept:

OWASP ZAP detected that the website uses HTTP rather than HTTPS.

Recommendation:-

Implement SSL/TLS encryption and enforce HTTPS for all communications.

5) Cookie Without HttpOnly Flag (Low)

Description	The application sets cookies without the **HttpOnly** attribute. Cookies without this attribute can be accessed by client-side scripts.
Impact	If a cross-site scripting attack occurs, attackers may steal session cookies and hijack user sessions.
System	http://127.0.0.1

Proof of concept:

OWASP ZAP detected cookies lacking the HttpOnly attribute in HTTP responses.

Recommendation:-

Configure cookies with the HttpOnly flag:**Set-Cookie: sessionid=value; HttpOnly**

6) Cookie Without SameSite Attribute (Low)

Description	The login credentials are visible to other users with username="test" password="test". It is visible to everyone who visits the signup page.
Impact	Critical
System	http://testphp.vulnweb.com/login.php

Proof of concept:

OWASP ZAP detected that cookies are missing the SameSite attribute.

Recommendation:

Configure cookies using the SameSite attribute:

SameSite=Strict or SameSite=Lax

7) X-Content-Type-Options Header Missing (Low)

Description	The **X-Content-Type-Options** header is missing from HTTP responses. This header prevents browsers from MIME-type sniffing.
Impact	Without this protection, attackers may trick the browser into executing malicious files disguised as legitimate content.
System	http://127.0.0.1

Proof of concept:

The OWASP ZAP scan identified the absence of the X-Content-Type-Options header.

Recommendation:-

Configure the following header:

X-Content-Type-Options: nosniff

8) Server Version Information Disclosure (Low)

Description	The web server reveals version information in HTTP response headers.
Impact	Attackers may use this information to identify known vulnerabilities associated with the server software version.
System	http://127.0.0.1

Proof of concept:

OWASP ZAP detected server version details in HTTP response headers.

Recommendation:-

Hide server version information in the web server configuration.
Example (Apache):
ServerTokens Prod
ServerSignature Off

9) Sitemap.xml Exposure (Medium)

Description	The file **sitemap.xml** was discovered during the scanning process. This file lists the URLs of the website and is commonly used by search engines.
Impact	The sitemap file may reveal internal or hidden URLs of the application, which could assist attackers in performing further reconnaissance.
System	http://127.0.0.1/sitemap.xml

Proof of concept:

OWASP ZAP detected the sitemap file during the crawling phase of the scan.

Recommendation:

Avoid including sensitive or administrative pages in the sitemap file and restrict access where necessary.

Security Weakness:

➢ Missing Security Headers

- During the assessment of the Damn Vulnerable Web Application (DVWA), several HTTP security headers were found to be missing or improperly configured. These include **Content Security Policy (CSP), X-Frame-Options, and X-Content-Type-Options** headers. The absence of these headers increases the risk of attacks such as cross-site scripting (XSS), clickjacking, and MIME-type sniffing.
- Implementing appropriate security headers can significantly strengthen the application's defense against common web attacks.

➢ Insecure Communication (HTTP)

- The web application is accessible through **HTTP instead of HTTPS**, which means that data transmitted between the client and the server is not encrypted. Attackers monitoring the network could intercept sensitive information such as login credentials or session cookies.
- Implementing **HTTPS using SSL/TLS encryption** would ensure that all communications between the user and the web server are secure and protected from interception.

➢ Cookie Security Misconfiguration

- The application was found to use cookies without proper security attributes such as **HttpOnly** and **SameSite**. These attributes help protect session cookies from being accessed by client-side scripts and reduce the risk of cross-site request forgery (CSRF) attacks.
- Proper cookie configuration is essential to prevent attackers from stealing or manipulating session information.

➢ Information Disclosure

- During the scanning process, files such as **sitemap.xml** and server information were publicly accessible. These files may reveal internal URLs, directory structures, and server details that could assist attackers during reconnaissance.
- Restricting access to sensitive files and hiding server version information can reduce the attack surface of the application.

➢ Directory Exposure

- The application allows directory browsing in certain locations. When directory listing is enabled, attackers may view the contents of directories and potentially access sensitive files.
- Disabling directory browsing and enforcing proper access control policies can help protect sensitive information from unauthorized access.

➢ Security Configuration Issues

- Several vulnerabilities detected during the assessment were related to **security misconfigurations**. These include missing security headers, exposed files, and weak server configurations. Security misconfigurations are one of the most common causes of web application vulnerabilities.
- Regular security assessments and proper server configuration management are recommended to reduce these risks.